Megh Chheda – I015

Anit Fadia – I020

Priya Gandhi – I022

OLA RIDE INSIGHTS

1. Introduction

Ride-hailing platforms generate large volumes of information about bookings, vehicle types, locations, customer activity, ride outcomes and recorded booking values. Analyzing this information helps describe usage patterns and operational challenges. Machine learning can also be used to test whether historical patterns provide useful signals for prediction.

OLA Ride Insights is an end-to-end academic machine learning project built using 103,024 OLA booking records dated 1–31 July 2024. The application combines five analytical modules: booking non-fulfilment prediction, booking-value estimation, customer segmentation, hourly demand forecasting and unusual-booking detection. The React and TypeScript dashboard communicates with a Python FastAPI backend, which reads the static dataset and saved machine-learning artifacts.

2. Problem Statement

Historical ride-booking data often remains difficult to interpret without structured analysis. Booking cancellations, changes in recorded booking values, varying ride demand, differences in customer behaviour and unusual records require different analytical methods. The project aims to develop a web-based system that cleans and explores OLA booking records, evaluates appropriate machine learning techniques against simple baselines, and presents results and limitations clearly through an interactive interface.

3. Objectives

- Prepare and explore historical ride-booking records.
- Predict the probability of an unfulfilled booking using information available at booking time.
- Estimate recorded booking values for completed rides.
- Identify customer groups using unsupervised learning.
- Forecast hourly booking volume and identify unusual booking records.
- Compare results with relevant statistical baselines and present findings in a usable website.

4. Dataset Description and Preparation

The supplied Bookings-100000-Rows.csv contains 103,024 rows and 20 original columns for July 2024. It includes booking status, vehicle type, pickup and drop locations, payment method, booking value, ride distance, cancellation reasons and customer/driver ratings. There are 94,544 distinct customers.

The data-processing pipeline removes the redundant Time and invalid Vehicle Images columns, parses dates, checks booking identifiers and creates calendar and outcome-based variables. It retains missing values when their absence is meaningful, for example where a cancelled ride has no payment method or rating. The dataset remains at 103,024 records after preprocessing. Different models then use different appropriate subsets and transformations.

Booking result	Bookings	Share
Successful	63,967	62.09%
Cancelled by driver	18,434	17.89%
Cancelled by customer	10,499	10.19%
Driver not found	10,124	9.83%

5. Methodology and Algorithms

Cancellation prediction uses Logistic Regression and Random Forest classification. Only features known when a booking is requested are used, including vehicle, route and time; post-outcome values are excluded to avoid data leakage. The target is an unfulfilled booking.

Fare estimation uses Linear Regression and Random Forest regression trained on 63,967 completed rides. Ride distance is treated as an estimated pre-booking route distance for prediction; the target is recorded Booking_Value. The values are experimental estimates rather than real-time fare quotations.

Customer segmentation uses K-Means clustering on aggregated behaviour for customers with at least two bookings. Demand forecasting compares a Random Forest using historical lag and calendar features with simple averages. Isolation Forest identifies statistically uncommon feature combinations and flags them for possible review, without concluding that they are fraudulent.

The offline training pipeline generates saved model files, evaluation metrics and processed outputs. The FastAPI application serves these to a React/TypeScript user interface. Training does not take place inside the deployed web server.

6. Model Evaluation and Results

The reported scores below come from the GitHub project evaluation summary. The models are assessed with metrics suitable for their tasks, not a single universal accuracy percentage.

Cancellation prediction

Method	Precision	Recall	F1	PR-AUC
Base-rate baseline	0.000	0.000	0.000	0.379
Logistic Regression	0.379	1.000	0.550	0.380
Random Forest	0.379	0.999	0.550	0.381

Random Forest is the selected model, but PR-AUC improves only slightly over the base rate. The available features have weak predictive power for cancellation.

Fare estimation

Method	MAE (INR)	RMSE (INR)	R^2

Global mean baseline	350.8	537.5	−0.0000
Vehicle-type mean	350.9	537.6	−0.0001
Linear Regression	351.0	538.4	−0.0030
Random Forest	351.7	538.5	−0.0037

Linear Regression is the selected machine learning model, but it does not outperform the global-mean baseline. Booking value is almost uncorrelated with ride distance ($r \approx 0.002$), indicating a significant dataset limitation for fare estimation.

Other analytical models

Task	Method / evaluation	Result
Customer segmentation	K-Means, k=7	Silhouette score 0.310; 7,954 customers
Demand forecast	Random Forest hourly forecast	MAE 10.05; RMSE 12.45
Demand baseline	Overall hourly mean	MAE 9.97; RMSE 12.56
Anomaly detection	Isolation Forest, 1% contamination	1,031 records flagged; stability 0.84

Customer clusters overlap; the hourly forecast offers mixed improvement compared with basic averages; and anomaly scores cannot be evaluated using ordinary accuracy without verified anomaly labels.

7. Key Findings and Discussion

- 62.09% of recorded bookings were successful, while 37.91% were unfulfilled.
- Driver cancellations accounted for 18,434 records, the largest category of unfulfilled bookings.
- Cancellation prediction showed very limited improvement over a base-rate approach.
- Booking value showed almost no relationship with recorded distance; fare-model R^2 values were negative.
- K-Means separated repeat customers into seven descriptive groups, but the silhouette score indicated overlapping segments.
- The anomaly model flagged 1,031 statistically unusual bookings for review, not confirmed fraud cases.

8. System Architecture

Data source (CSV) → Python/pandas preparation → feature engineering → model-specific training using scikit-learn → evaluation and saved joblib/JSON/Parquet artifacts → FastAPI prediction and analytics endpoints → React, TypeScript, Tailwind and Recharts dashboard → user-facing insights. The current deployment uses a static dataset and saved models rather than a live transaction database.

9. Conclusion

OLA Ride Insights demonstrates an end-to-end implementation of machine learning for mobility data, spanning cleaning, descriptive analysis, supervised prediction, unsupervised learning, model comparison and web deployment. Its main contribution is the integration of several analytical techniques into one interface with transparent evaluation. The experiments also

demonstrate why weak predictive performance must be reported instead of presenting every trained model as accurate or operationally reliable.

10. Future Scope

- **Richer datasets:** Collect verified trip-level pricing, traffic, weather, supply and demand information over multiple months.
- **Improved prediction:** Try feature engineering, calibration, cross-validation and alternative regression and classification approaches after strengthening the input data.
- **Longer-horizon demand forecasting:** Use larger historical windows to evaluate weekly, seasonal and event-related demand changes.
- **Validated anomaly detection:** Add manually reviewed anomalous and normal examples to measure precision and false positives.
- **Real-time application support:** Integrate live feeds and periodic model retraining, with privacy-aware monitoring and performance checks.
- **Enhanced visualization:** Add geographic analyses and clearer explanations of uncertainty in the dashboard.

11. References

Project GitHub repository: https://github.com/meghchheda/Ola-Fare-Analysis

Dataset in project repository: https://github.com/meghchheda/Ola-Fare-Analysis/tree/main/data

Model evaluation report: https://github.com/meghchheda/Ola-Fare-Analysis/blob/main/docs/model_evaluation_summary.md

Live frontend website: https://ola-fare-analysis.vercel.app/

Backend API deployment: https://ola-fare-analysis-1.onrender.com/

Original Kaggle upload: https://www.kaggle.com/datasets/muhammadahmadmujahid/ola-dataset